**Program:** BS Computer Science
**Semester:** Fall 2025
**Credit Hour(s):** 1
**Pre-requisite(s):** EE1005 Digital Logic Design
**Instructor:** Engr. Muhammad Hassaan Bin Shoukat – hassaan.shoukat@nu.edu.pk

# Lab # 12: Printing Numbers, Calculating Screen Locations

## 1. Objectives

- Understand how to convert binary numbers to ASCII for display.
- Use the DIV instruction to decompose numbers.
- Calculate specific video memory addresses using the MUL instruction.

## 2. Printing String

Printing a string in assembly language primarily involves transferring a sequence of ASCII characters from the data segment to the video memory segment, which starts at address 0xB800. Since video memory is mapped directly to the screen, writing data to this specific range immediately displays it to the user. The process typically uses a loop where the SI (Source Index) register points to the string in memory and the DI (Destination Index) register points to the specific screen location.

**Example:** Print a String Using Subroutine

```
[org 0x0100]
jmp start

message: db 'Assembly Rocks!', 0
length:  dw 15

printstr:
    push bp
    mov  bp, sp
    push es
    push ax
    push cx
    push si
    push di

    mov  ax, 0xb800
    mov  es, ax
    mov  si, [bp+6]
    mov  cx, [bp+4]
```

```
    mov  di, 0
    mov  ah, 0x0E
nextchar:
    mov  al, [si]
    mov  [es:di], ax
    add  di, 2
    add  si, 1
    loop nextchar

    pop  di
    pop  si
    pop  cx
    pop  ax
    pop  es
    pop  bp
    ret  4
start:
    mov  ax, message
    push ax
    push word [length]
    call printstr
mov  ax, 0x4c00
int  0x21
```

## 3. Number Printing

In Assembly, numbers are stored as binary values. If you try to print the number 10 directly to video memory, the computer will print the character associated with ASCII code 10 (a control character), not the digits "1" and "0".

**The Algorithm:**

i. *Divide* the number by the base (10 for decimal)
ii. The *Remainder* is the rightmost digit. Convert it to ASCII (Add 0x30)
iii. *Push* the remainder onto the *Stack* (because digits are extracted backwards: Right to Left)
iv. Repeat until the quotient is 0
v. *Pop* from the Stack and print digits Left to Right

**Conceptual Examples:**

**1.** Print **253**

| Step | Dividend | / 10 | Quotient | Remainder → Digit |
|---|---|---|---|---|
| 1 | 253 | /10 | 25 | 3 |
| 2 | 25 | /10 | 2 | 5 |
| 3 | 2 | /10 | 0 | 2 |

Stack (top → bottom): 2, 5, 3
Output: **253**

**2.** If base = 2, push remainders: 13 → 1101

**Example 1:** Print a Single Digit (0–9 Only)

```
[org 0x0100]

mov ah, 0x0E      ; BIOS teletype output
mov al, '7'       ; character to print
int 0x10          ; print it

mov ax, 0x4C00
int 0x21
```

**Example 2:** Print Any Two-Digit Number (10–99)

```
[org 0x0100]

mov ax, 57        ; number to print
mov bl, 10
div bl       ; AH = units, AL = tens (AL= quotient, AH=remainder)

mov ah, 0x0E   ; BIOS video
add al, 30h       ; convert to ASCII
int 10h

; print units digit
mov al, ah        ; remainder
add al, 30h
int 10h

mov ax, 4C00h
int 21h
```

**Example 3:** Print Any 3-Digit NUMBER (100–999)

```
[org 0x0100]

mov ax, 345       ; number to print
mov bx, 100
div bx            ; AL = hundreds, DX = remainder

; print hundreds digit
mov ah, 0x0E
add al, 30h
```

```
int 10h

mov ax, dx          ; remainder
mov bl, 10
div bl              ; AL = tens, AH = units

; print tens digit
mov ah, 0x0E
add al, 30h
int 10h

; print units digit
mov al, ah
add al, 30h
int 10h

mov ax, 4C00h
int 21h
```

**Example 4:** General Number Printing. Works for ANY 16-bit number (0–65535).

```
[org 0x0100]

mov ax, 4529
call printnum

mov ax, 4C00h
int 21h

printnum:
    mov bx, 10
    xor cx, cx

next_digit:
    xor dx, dx
    div bx              ; AX/BX
    add dl, 30h         ; digit ASCII
    push dx
    inc cx
    test ax, ax
    jnz next_digit

print_loop:
    pop ax
    mov ah, 0x0E
```

```
    int 10h
    loop print_loop
    ret
```

**Example 5:** General Number Printing Subroutine (Stack Based)

```
[org 0x0100]
jmp start
clrscr:
    push es
    push ax
    push cx
    push di

    mov ax, 0B800h
    mov es, ax
    mov ax, 0720h           ; space with normal attribute
    mov cx, 2000            ; 80 * 25 = 2000 cells
    xor di, di
    rep stosw

    pop di
    pop cx
    pop ax
    pop es
    ret

printnum:
    push bp
    mov bp, sp

    push ax
    push bx
    push cx
    push dx
    push es
    push di
    mov ax, 0B800h
    mov es, ax
    mov ax, [bp+4]      ; number to print
    mov bx, 10          ; base = decimal
    mov cx, 0           ; digit counter

next_digit:
    xor dx, dx          ; must clear DX before div
```

```
    div bx              ; AX = quotient, DX = remainder
    add dl, 30h         ; convert remainder → ASCII
    push dx             ; store digit
    inc cx
    cmp ax, 0
    jnz next_digit

    mov di, 0           ; print at top-left
print_loop:
    pop dx
    mov dh, 07h         ; attribute
    mov [es:di], dx
    add di, 2
    loop print_loop

    pop di
    pop es
    pop dx
    pop cx
    pop bx
    pop ax
    pop bp
    ret 2               ; remove parameter from stack
start:
    call clrscr

    mov ax, 4529
    push ax
    call printnum
mov ax, 4C00h
int 21h
```

**Example 6:** Print in Binary (Base 2)

```
[org 0x0100]
jmp start

clrscr:
    push es ax cx di
    mov ax, 0B800h
    mov es, ax
    mov ax, 0720h
    mov cx, 2000
    xor di, di
    rep stosw
```

```
    pop di cx ax es
    ret

printnum:
    push bp
    mov bp, sp
    push ax bx cx dx es di

    mov ax, 0B800h
    mov es, ax

    mov ax, [bp+4]      ; number
    mov bx, [bp+6]      ; base (2 or 10 or 8)
    mov cx, 0
next_digit:
    xor dx, dx
    div bx
    add dl, 30h         ; convert to ASCII
    push dx
    inc cx
    cmp ax, 0
    jnz next_digit

    mov di, 0
print_loop:
    pop dx
    mov dh, 07h
    mov [es:di], dx
    add di, 2
    loop print_loop

    pop di es dx cx bx ax
    pop bp
    ret 4
start:
    call clrscr
    mov ax, 13          ; number → 13 = 1101 in binary
    mov bx, 2           ; base = 2
    push bx
    push ax
    call printnum
mov ax, 4C00h
int 21h
```

**Example 7:** Printing in Octal (Base 8)

By changing the divisor (Base) in BX, we can print in other number systems. Here we print 100 in Octal. Decimal 100 = Octal 144

```
;mov bx, 10      <-- Change to -->      mov bx, 8

start:
    mov ax, 100      ; Number to convert
    push ax
    call printnum    ; Will print '144' on screen
mov ax, 0x4c00
int 0x21
```

**Example 8:** Arithmetic and Printing**.** This example adds two numbers and prints the result.

```
start:
    mov ax, 150
    mov bx, 50
    add ax, bx       ; AX now holds 200

    push ax          ; Push the result (200)
    call printnum    ; Calls the standard base-10 routine
mov ax, 0x4c00
int 0x21
```

## 4. Screen Location Calculation

Video memory is a linear array, but the screen is a 2D grid (80 columns x 25 rows). To find the memory address (offset) for a coordinate (x, y):

Offset = **(Row * 80 + Col) * 2**

**Example 9:** "Hello World" at (30, 20). Using MUL to calculate the exact offset.

```
[org 0x0100]
jmp start
message: db 'Hello World', 0
msg_len: dw 11

printstr:
    push bp
    mov  bp, sp
    push es
    push ax
    push cx
    push si
    push di
```

```
    mov  ax, 0xb800
    mov  es, ax

    ; Calculation: (row * 80 + column) * 2
    mov  al, 80
    mul  byte [bp+10]       ; Multiply AL by row
    add  ax, [bp+12]        ; Add column
    shl  ax, 1              ; Multiply by 2 (Shift Left)

    mov  di, ax             ; DI points to calculated location
    mov  si, [bp+6]         ; Source string address
    mov  cx, [bp+4]         ; Length
    mov  ah, [bp+8]         ; Attribute
nextchar:
    lodsb                   ; Load byte from SI to AL, inc SI
    stosw                   ; Store AX to ES:DI, inc DI by 2
    loop nextchar

    pop  di
    pop  si
    pop  cx
    pop  ax
    pop  es
    pop  bp
    ret  10
start:
    mov ax, 30
    push ax                 ; column = 30
    mov ax, 20
    push ax                 ; row = 20
    mov ax, 0x07            ; Attribute = Normal
    push ax
    mov ax, message
    push ax                 ; String Address
    push word [msg_len]     ; Length
    call printstr
    mov ah, 0x0
    int 0x16
mov ax, 0x4c00
int 0x21
```

**Example 10:** Center of Screen Character. Directly calculating the center (40, 12) without a subroutine to show the math clearly.

```
[org 0x0100]
    mov ax, 0xb800
    mov es, ax

    ; Calculate Offset for (40, 12)
    mov ax, 12          ; Y = 12
    mov bl, 80
    mul bl              ; AX = 12 * 80 = 960
    add ax, 40          ; AX = 960 + 40 = 1000
    shl ax, 1           ; AX = 1000 * 2 = 2000

    mov di, ax
    mov word [es:di], 0x0741 ;Print 'A'(41h) with attribute 0x07

    mov ah, 0x0
    int 0x16
mov ax, 0x4c00
int 0x21
```

**Example 11:** Drawing a Vertical Line.

```
[org 0x0100]
    mov ax, 0xb800
    mov es, ax
    mov di, 100         ; Start somewhat near top left
    mov cx, 15          ; Draw line 15 characters long
    mov ax, 0x077C      ; 0x7C is ASCII for '|'

draw_line:
    mov [es:di], ax     ; Draw pipe character
    add di, 160         ; Move EXACTLY one row down (80*2)
    loop draw_line

    mov ah, 0x0
    int 0x16
mov ax, 0x4c00
int 0x21
```

## 6. Lab Tasks

**Task 1:** Write an 8088 assembly program that:

a) Takes a string (your name) from memory
b) Computes the correct video memory location
c) Calls a *custom print-string subroutine* to display the name at the calculated screen position.

*Restrictions:*

- Must use *MUL* for row computation
- Must not use BIOS teletype (int 10h, AH=0Eh)
- Must write directly to *0xB800:DI*

**Task 2:** Write a program that:

a) Stores a 16-bit number in AX (any value you choose).
b) Converts it to *four hexadecimal ASCII digits* using repeated division.
c) Prints the HEX result on the top row starting at column 5.
d) Must implement your own *hex-digit-to-ASCII converter*.
e) Must print digits in correct left-to-right order using a stack.

*Restrictions:*

- No hard-coded ASCII array allowed.

**Task 3:** Write a program that:

a) Draws a hollow box on the screen using the '#' character.
b) The rectangle must be:
    - Top-left corner at (10, 5)
    - Width = 30 characters
    - Height = 8 characters
c) Use:
    - **STOSW** with REP for drawing horizontal lines
    - Calculated DI offsets for each row boundary
    - Manual DI offset calculation (only MUL allowed for row computations)

*Restrictions:*

- Only borders should appear—inside must remain unchanged.